

You used:

 ChromaDB

Flow:

PDF → Text → Embeddings → Stored in ChromaDB

Then:

Question → Embedding → Search similar chunks

"I used ChromaDB as a vector database to store embeddings of PDF chunks and perform semantic search before passing context to the LLM."

TECHNICAL ROUND ANSWERS

"Walk me through your project."

"I built StudyMind—a personal knowledge agent for students. The idea came from a real problem I faced: I had notes scattered across PDFs and folders but no way to query across them. ChatGPT would answer my questions but hallucinate—giving confident answers not from my actual material.

I realized this is exactly what Atlan solves for enterprises. Data is scattered everywhere, and AI reasoning is on unverified context. So I built a context layer for students—you upload your PDFs, it indexes them into a vector database using embeddings, and when you ask a question, it answers only from your actual notes with source citations. No hallucination possible because it can't answer outside its given context."

"What is an embedding?"

"An embedding is a list of numbers that represents the meaning of a piece of text mathematically. Similar meanings produce similar numbers. So when I search for 'machine learning,' it finds chunks about 'ML algorithms' even if those exact words don't appear—because the meaning is similar. This is called semantic search—understanding meaning rather than matching keywords."

"Why ChromaDB?"

"Three reasons. It runs completely locally — no data leaves the machine, which matters for private student notes. It's free with no API calls needed for storage. And it has a simple

Python API that let me move fast. The tradeoff is it doesn't scale to millions of documents, but for a student project that's perfectly fine."

"Why do you chunk text into 500 words?"

"Two reasons. First, LLMs have a context window limit — you can't send an entire textbook in one prompt. Second, smaller chunks make retrieval more precise. If I search for one concept I get the specific paragraph about it, not an entire chapter. 500 words was a judgment call—small enough for precision, large enough to preserve context within a chunk."

"What is RAG, and does your project use it?"

"RAG stands for Retrieval Augmented Generation. Instead of relying only on the LLM's training data, you retrieve relevant documents and inject them into the prompt as context before asking the question. Yes, StudyMind is entirely RAG-based. The retrieval is ChromaDB finding similar chunks. The augmentation is injecting those chunks into the Gemini prompt. The generation is Gemini, answering from that context only."

"What is a vector database?"

"A vector database stores embeddings — numerical representations of text meaning — and lets you search them by similarity rather than exact match. ChromaDB is the vector database I used. When I store a note chunk, I store its embedding alongside it. When I search, I convert the query to an embedding and find the closest stored embeddings. That's how it finds semantically relevant content even without keyword matches."

"What prevents the AI from hallucinating in your project?"

"Two mechanisms. First, the prompt explicitly instructs Gemini to answer only from the provided note excerpts and to say, 'I couldn't find this in your notes' if the context doesn't contain relevant information. Second, the retrieval step means Gemini never sees anything except the top 3 matching chunks from my actual notes—it has no opportunity to draw from its general training. Grounding is the architectural solution to hallucination."

"What is a system prompt?"

"A system prompt is the instruction you give an LLM before the conversation starts — it sets how the model should behave, what constraints it should follow, and what role it should play. In StudyMind the system prompt tells Gemini it's a study assistant that must answer only

from provided excerpts. This is the single most important line of code for preventing hallucination."

"What is the context window?"

"The maximum amount of text an LLM can process in one request. Everything outside the context window is invisible to the model. This is why chunking matters—if your notes are 100 pages but the context window fits 3 pages, you need to retrieve only the relevant 3 pages rather than sending everything."

"What happened when you switched libraries mid-build?"

"The google-generativeai library got deprecated while I was building. I had to switch to the new google-genai library, which had different import structures, different client initialization, and a different embedding model name. It broke the project temporarily. I debugged by reading the error messages carefully, checked the new library's documentation, and rewrote the affected files. It taught me to always check library maintenance status before choosing a dependency—something that matters a lot at an enterprise scale too."

"What is data lineage, and how does your project relate to it?"

"Data lineage tracks where data came from and how it flows through systems. In Atlan's world that means tracing a dashboard metric back through transformations to the original source table. In StudyMind every answer includes the source citation—which PDF, which chunk the answer came from. That's lineage for knowledge. You can trace every answer back to its source, just like Atlan traces every data point back to its origin."

"What is the difference between fine-tuning and RAG?"

"Fine-tuning bakes knowledge into the model's weights permanently through additional training. RAG fetches relevant context at runtime and injects it into the prompt. For enterprise data that changes constantly — like student notes that update every semester — RAG is far more practical. You don't retrain a model every time notes change. You just update the vector database. Fine-tuning is expensive, slow, and inflexible. RAG is fast, cheap, and always current."

"Why did you use Gemini and not OpenAI?"

"Gemini's API has a generous free tier, which made it practical for a student project with zero budget. The API structure is similar to OpenAI's, so the concepts transfer directly. The

tradeoff is OpenAI's models are generally considered more capable, but for a proof-of-concept demonstrating the architecture, Gemini was the right call."

"What would you improve if you had more time?"

"Three specific things. First — sliding window chunking with overlap so context isn't lost at chunk boundaries. Right now a sentence cut in half between two chunks loses meaning. Second, an automated evaluation system to score answer quality, because I currently have no way to measure whether answers are actually good. Third—a web UI instead of a terminal so it's usable by real students, not just developers."

"How does your project relate to what Atlan does?"

"The parallel is almost exact. Atlan takes enterprise data scattered across Snowflake, dbt, and Tableau—extracts metadata; builds a context layer; and lets AI agents query it with trust. I took student notes scattered across PDFs, extracted text, built a context layer in ChromaDB, and let Gemini query it with grounding. The problem is identical, just at different scales. Atlan's context layer serves Mastercard's 18 million data assets. Mine serves a student's 10 PDFs. Same architectural pattern."

"Did you use AI to build this project?"

"Yes, completely honestly—I used AI as a coding partner, similar to how engineers use GitHub Copilot or Cursor daily. I don't have deep Python experience yet. But I understand every single line of this code, every design decision, and every tradeoff. I can walk you through any part of it right now. More importantly—building this taught me things I couldn't have learned just reading. The hardest part wasn't the AI at all. It was the data ingestion layer—PDF extraction is messy, chunking strategy affects answer quality, and a library I started with got deprecated mid-build. That's a real engineering lesson."

CULTURE ROUND ANSWERS

"Tell me about yourself and why you applied."

"I'm a B.Tech student graduating in 2027. I've spent the last few weeks genuinely trying to understand what Atlan builds — not just the product but the problem. The context layer problem resonated with me: 95% of AI pilots fail not because the models are bad but because AI doesn't understand what the data means. That's the actual unsolved problem. I wanted to work somewhere the problem itself is interesting, not just the technology. And I wanted to prove to myself I could build something real — so I did."

"What problem genuinely energizes you?"

"I get obsessed with the gap between AI's potential and its actual reliability. AI can be incredibly capable but the moment it reasons on bad or missing context, it confidently gives wrong answers. That failure mode — confident wrongness — is what interests me. StudyMind was my attempt to solve it at a small scale. Atlan is solving it at enterprise scale. That gap between potential and reliability is where I want to work."

"What would you do in the first week if your manager gave you no task?"

"First I'd spend two days reading everything — docs, recent PRs, customer conversations, internal discussions. I'd want to understand what's actually being built and what's actually broken before touching anything. Then I'd pick the smallest meaningful problem I could find, make an assumption about the right solution, state that assumption clearly, build a draft, and get feedback within the week. I wouldn't wait for definition. I'd create it and be wrong fast."

"Tell me about a time you were wrong and how you handled it."

"During this project I assumed the PDF extraction would be clean and straightforward. It wasn't. Text came out garbled, pages were out of order, some PDFs returned nothing. My initial chunking strategy broke completely on messy input. I had to go back, add error handling, test on multiple different PDF types, and rethink the chunk size. I was wrong about how clean real data would be. That's probably the most important thing I learned — real data is always messier than you expect. Atlan's customers deal with this at massive scale."

"How do you work when the spec is missing?"

"I make an assumption, state it explicitly, and ship something. Waiting for perfect clarity is the worst option — you learn nothing and deliver nothing. I'd rather put a draft in front of someone and be told I got it wrong than polish something in private that solves the wrong problem. The faster you're wrong, the faster you're right."

"What do you not know yet that matters for this role?"

"Honestly — a lot. I don't know dbt deeply. I haven't worked with Snowflake or Databricks at all. I've never shipped to production in a real engineering team. And my Python experience is still early. What I do have is the ability to learn fast, a genuine understanding of the problem Atlan is solving, and one real project that proves I can figure things out without being told exactly what to do. I think those matter more at this stage than checklist knowledge."

"Why Atlan specifically?"

"Most AI companies are building better models. Atlan is building the infrastructure that makes models actually useful inside enterprises. The context layer problem — making AI understand what data means, not just what it says — that's the unsolved problem. And it maps directly to what I built. I wanted to work somewhere the problem itself is interesting, not just the technology. When I read that 95% of AI pilots fail because of missing context — not bad models — that sentence reframed everything for me. That's the problem I want to work on."

"Where do you see yourself after this internship?"

"Honestly — I want to earn a full-time offer. Not because it's a stepping stone but because I genuinely want to keep working on this problem. The context layer for AI is infrastructure that will matter for the next decade. I want to be someone who helped build it from early on."

THE GOLDEN RULE FOR ALL ANSWERS

Don't memorize these words word for word. Read them 3-4 times, understand the logic behind each answer, and then say it in your own natural voice.

The interviewer will feel immediately if you're reciting vs. actually thinking. The ideas matter more than the exact words.

You're ready. Go get it. 🚀

The Right Way to Handle It

Be completely upfront — but frame it the way real engineers actually work.

Say this:

"I'll be honest — I used AI as a coding partner to build this, similar to how engineers use GitHub Copilot or Claude daily. I don't have deep Python experience yet. But I understand every single line of this code, every design decision, and why it was built this way. I can walk you through any part of it right now."

Then actually walk them through it confidently.

That answer is **honest, self-aware, and shows exactly the AI-native mindset they're hiring for**. Remember — the JD literally says "AI-native by default." Using AI to build is not a weakness here. It's evidence.

But You MUST Be Able to Defend Every Line

This is the real requirement. You can't just say "I used AI" and stop there. You need to own the code completely.

So let's go through every important part right now so you understand it deeply.

Understanding Your Own Code

ingest.py — What It Does Line by Line

python

```
client = genai.Client(api_key="...")
```

Creates a connection to Google's Gemini API. Like logging into a service before using it.

```
db_client = chromadb.PersistentClient(path="./db")
```

```
collection = db_client.get_or_create_collection(name="studymind")
```

Creates a local database folder called `db` on your machine. The collection is like a table inside that database — where all your note embeddings live.

python

```
def extract_text_from_pdf(pdf_path):
```

Opens a PDF file, reads it page by page, and extracts raw text. PyPDF2 does the heavy lifting here.

```
def chunk_text(text, chunk_size=500):
```

Takes a wall of text and splits it into 500-word pieces. Why? Because LLMs have context limits — you can't send 50 pages at once. Smaller chunks also make searching more precise.

python

```
embedding = get_embedding(chunk)
```

This is the most important line. Takes a chunk of text and converts it into a list of numbers—called an embedding—that represents the *meaning* of that text mathematically. Similar meanings = similar numbers.

python

```
collection.add(documents, embeddings, metadatas, ids)
```

Stores the chunk, its embedding, and where it came from (filename, chunk number) into ChromaDB.

agent.py — What It Does Line by Line

python

```
def search_notes(query, n_results=3):
```

Takes your question, converts it to an embedding, then finds the 3 most similar chunks in ChromaDB using vector similarity. Like a semantic Google search over your notes.

python

```
results = collection.query(query_embeddings=[embedding], n_results=n_results)
```

This is the actual search. ChromaDB compares your question's embedding against all stored embeddings and returns the closest matches.

python

```
context = "\n\n".join(context_parts)
```

Combines the top matching chunks into one block of text — this becomes the "context" the AI reads before answering.

```
prompt = f"""You are a helpful study assistant. Answer ONLY using the provided excerpts... """
```

This is the system prompt — the instructions that tell Gemini how to behave. The key instruction is "ONLY from the "notes"—this is what prevents hallucination.

python

```
response = client.models.generate_content(model="gemini-1.5-flash", contents=prompt)
```

Sends the prompt to Gemini and gets the answer back.

5 Questions They'll Ask About Your Code

"Why did you use ChromaDB specifically?"

"It runs completely locally — no data leaves my machine. For a student's private notes that matters. It's also free and has a simple Python API that made it fast to get started. The tradeoff is it doesn't scale to millions of documents, but for a student project that's fine."

"What is an embedding and why do you need it?"

"An embedding is a list of numbers that represents the meaning of a piece of text. Words or sentences with similar meanings get similar numbers. So when I search for 'machine learning definition,' it finds chunks about 'ML algorithms' even if those exact words aren't there. That's semantic search—understanding meaning, not just matching keywords."

"Why do you split text into chunks?"

"Two reasons. First, LLMs have a context window limit — you can't send an entire textbook in one prompt. Second, smaller chunks make retrieval more precise. If I search for one concept, I get the specific paragraph about it, not an entire chapter."

"What happens if the answer isn't in the notes?"

"The prompt explicitly tells Gemini to say 'I couldn't find this in your notes' if the retrieved chunks don't contain relevant information. That's the grounding mechanism—without it, Gemini would fall back to its general training and potentially hallucinate."

"What would you improve if you had more time?"

"Three things. Sliding window chunking with overlap so context isn't lost at chunk boundaries. An eval system to score answer quality automatically and a web UI instead of a terminal so it's actually usable by real students."

The One Thing That Will Convince Them

At some point just say this naturally:

"Building this taught me something I didn't expect — the hardest part wasn't the AI at all. It was getting clean data in. PDF extraction is inconsistent, chunking strategy affects answer quality, and the library I started with got deprecated mid-build. That's probably true at Atlan's scale too—the integration and data layer is always messier than it looks from outside."

That one insight — learned from actually building — shows you're a real engineer, not someone who just ran a tutorial.

1. General Motors — Transparent AI

What happened: GM's Data Governance Lead chose Atlan specifically because of its API infrastructure—they needed to do things in an automated way, getting anything in or out as needed, embedding directly into their engineering processes. [atlan](#)

The core problem GM had: they were building AI systems but couldn't make them transparent—nobody could trace *why* the AI made a decision or *which data* it used. Without lineage and governance, AI answers couldn't be trusted internally.

How to use this in your interview:

"GM's story resonated with me when building StudyMind. They couldn't trust their AI because there was no transparency about which data it used. I had the same problem—ChatGPT would answer my study questions, but I had no idea if it was hallucinating or actually from my notes. So I built source citations into every answer, like lineage for student knowledge. Same problem, different scale."

How to Tell This Story in the Interview

When They Say "Walk Me Through Your Project"

Don't start with tech. Start with the problem:

"I was preparing for this internship and realized I had 20 tabs, 5 PDFs, and 3 sets of notes — but no way to query across them. I'd ask ChatGPT, and it would give me confident answers that weren't actually in my course material. That's a hallucination—and it's dangerous when you're studying for exams."

Then I realized this is exactly the problem Atlan solves for enterprises. Data teams have the same issue. Data is scattered everywhere, AI agents are reasoning on bad or missing context, and there is no single trusted source of truth.

So I built StudyMind — a personal context layer for students. You upload your notes, it indexes them into a vector database, and when you ask a question it answers ONLY from your actual content—with citations showing exactly which note it came from."

When They Ask "How Does It Work Technically?"

Explain it in layers—start simple, go deeper only if they ask:

Layer 1—Simple: *"You upload PDFs. It extracts the text, splits it into chunks, converts each chunk into an embedding—a mathematical representation of meaning—and stores it in ChromaDB locally. When you ask a question, your question gets converted to an embedding too, and we find the most similar chunks using vector similarity search. Those chunks become the context for Gemini to answer from."*

Layer 2—If they dig deeper, *"The key design decision was keeping the vector database local with ChromaDB. No external API calls for storage, no data leaving your machine. For embeddings I used Gemini's embedding-001 model. The retrieval uses cosine similarity under the hood—ChromaDB handles that automatically."*

Layer 3—If they go even deeper: *"The chunking strategy matters a lot. I split by word count—500 words per chunk—with the tradeoff being that smaller chunks are more precise but lose surrounding context. A next improvement would be sliding window chunking with overlap so context isn't lost at chunk boundaries."*

When They Ask "What Would You Improve?"

This is a trap most candidates fail—they say "nothing" or give vague answers. Have specific answers ready:

"Three things. First — sliding window chunking with overlap to preserve context at boundaries. Second—a proper evaluation system to score answer quality, because right now I have no way to measure if the answers are actually good. Third—a web UI instead of terminal, so it's more usable for actual students."

This answer shows you think like an engineer, not just a student who built a toy.

When They Ask "How Is This Related to What Atlan Does?"

This is your moment — deliver this cleanly:

"The parallel is almost exact. Atlan takes enterprise data scattered across Snowflake, dbt, Tableau — extracts metadata, builds a context layer, and lets AI agents query it with trust. I took student notes scattered across PDFs — extracted text, built a context layer in ChromaDB, and let Gemini query it with grounding. The problem is identical, just at different scales. Atlan's context layer is for Mastercard's 18 million data assets. Mine is for a student's 10 PDFs. Same architectural pattern."

When They Ask "What Did You Learn Building This"

Be honest and specific:

"Three things surprised me. One—the hardest part wasn't the AI; it was getting the data in cleanly. PDF extraction is messy and inconsistent. That's probably true at an enterprise scale too—the integration layer is always harder than it looks. Two—I learned why grounding matters so much. Without it, the AI confidently answers from its training data, not your notes. That's exactly why Atlan's context layer exists. Three — I switched from the old google-generativeai library to google-genai mid-build because the old one was deprecated. That taught me to always check library maintenance status before choosing dependencies."

That last point is gold — it's a real engineering lesson that shows maturity.

When They Ask "Why Atlan Specifically"

Don't say "I like AI" or "great company." Say this:

"Most AI companies are building better models. Atlan is building the infrastructure that makes models actually useful inside enterprises. The context layer problem—making AI understand what data means, not just what it says—that's the unsolved problem. And it maps directly to what I built. I wanted to work somewhere the problem itself is interesting, not just the technology."

Things to Learn Before You Start

You don't need to know everything, but getting familiar with these will help a lot:

- **What a data catalog is** (you now know the basics)
- **What metadata and data lineage mean** — key Atlan concepts
- **MCP (Model Context Protocol)** — how AI tools plug into data sources
- **Basic data stack tools**: dbt, Snowflake, Tableau—Atlan connects to all of these
- **Atlan's blog and docs** at atlan.com — very readable and product-focused

Study Plan (2–3 weeks before joining)

Week 1 — Understand the Domain

- Day 1–2: Understand what a data catalog is and why companies need one. Read Atlan's homepage and their "What is a data catalog" page.
- Day 3–4: Learn what metadata, data lineage, and data governance mean (explained below).
- Day 5–7: Watch a few YouTube videos on the modern data stack — search "modern data stack explained."

Week 2 — Understand Atlan's Tech

- Day 1–2: Read Atlan's blog — they write really well about AI + data topics.
- Day 3–4: Learn what MCP (Model Context Protocol) is and how AI agents use it.
- Day 5–7: Get hands-on — sign up for Atlan's free trial or explore their demo environment if available.

Week 3 — AI-Native Builder Skills

- Day 1–3: Build something small with the Anthropic or OpenAI API — even a simple chatbot that answers questions from a data source.
- Day 4–5: Learn basic SQL if you don't know it — Atlan's world is deeply SQL-adjacent.
- Day 6–7: Read about agentic AI workflows — how AI agents take actions, not just answer questions.

Key Concepts Explained Simply

Data Catalog: Imagine a company has 10,000 data tables across 50 systems. Nobody knows which table has the right revenue numbers, who owns it, or if it's outdated. A data catalog is the index that answers all of this—like a library catalog, but for data.

Metadata: Data *about* data. The table name, who created it, when it was last updated, and what each column means—that's all metadata. Atlan's whole job is managing metadata at enterprise scale.

Data lineage tracks where data came from and where it goes. If a CEO's dashboard shows wrong revenue, lineage lets you trace backward—from dashboard → report → transformation → raw table → source system—to find the bug. Atlan does this automatically and visually.

Data governance rules are about who can access what data, who's responsible for it, and whether it's trustworthy. Think of it as the "policies and ownership" layer on top of data. Especially important for compliance (GDPR, HIPAA, etc.).

MCP (Model Context Protocol) A standard that lets AI tools (like Claude or ChatGPT) plug into external data sources and act on them. Atlan acts as an MCP server, so when an AI agent needs to know, "What does the `revenue` column mean in our data warehouse?" — it queries Atlan in real time instead of guessing.

The Modern Data Stack A collection of tools companies use together: Snowflake or BigQuery (store data), dbt (transform data), Airflow (orchestrate pipelines), and Tableau or Looker (visualize data). Atlan sits *above* all of these as the context and governance layer connecting them.

Agentic AI: AI that doesn't just answer questions—it takes actions. An AI agent might autonomously find the right dataset, check its quality, query it, and generate a report. Atlan is building the infrastructure for these agents to work safely inside enterprises.

The Concept Map (Learn in This Order)

Layer 1 — How AI Actually Works (3 days)

These are the absolute basics. Without these, nothing else makes sense.

1. What is an LLM? A large language model. Understands and generates text by predicting the next word, trained on massive data. Key insight: it has no memory between conversations and no access to real-time data by default.

2. What is a prompt? The input you give an LLM. Prompt engineering is the skill of structuring that input to get better outputs. You'll use this constantly.

3. What is context window? The amount of text an LLM can "see" at once. Everything outside the context window — it doesn't know. This is why context management matters so much.

4. What is RAG? (Retrieval Augmented Generation) Instead of the LLM relying only on its training, you fetch relevant documents and inject them into the prompt. Like giving the AI a cheat sheet before it answers. Atlan's whole value prop is essentially RAG done right for enterprise data.

5. What is an AI agent? An LLM that doesn't just answer — it takes actions. It can call tools, search databases, write files, make decisions in a loop. Think of it as an LLM with hands.

6. What are tools/function calling? How agents take actions. You define a function like `get_table_schema()` and tell the LLM it can call it. The LLM decides when to use it. This is foundational to everything agentic.

Layer 2 — The Data World (3 days)

Atlan lives here. You need enough to not be lost.

7. What is a data warehouse? A central place where a company stores all its structured data. Snowflake, BigQuery, Databricks are the big ones. Think of it like a giant organized spreadsheet factory.

8. What is SQL? The language used to query data warehouses. Even basic SELECT, WHERE, JOIN knowledge helps enormously. Spend one day on this minimum.

9. What is dbt? A tool that transforms raw data in warehouses into clean, usable tables. Atlan integrates deeply with dbt. You don't need to use it, just know what it does.

10. What is a data pipeline? The journey data takes from source → storage → transformation → dashboard. Airflow orchestrates these pipelines. Again, just know the concept.

11. What is metadata? Data about data. Column names, owners, descriptions, last updated time — all metadata. This is Atlan's entire business.

Layer 3 — Where AI Meets Data (3 days)

This is your role's sweet spot.

12. What is grounding? Making an AI's answer based on real, specific data rather than its general training. The difference between ChatGPT hallucinating a revenue number vs reading your actual database.

13. What is hallucination? When an LLM confidently says something wrong. In consumer apps it's annoying. In enterprise data it's catastrophic — imagine a wrong revenue figure in a Mastercard dashboard.

14. What is a context layer? Atlan's term. The shared layer of metadata, definitions, lineage, and governance that both humans and AI agents can query to understand what data means. The "missing link" between raw data and trusted AI answers.

15. What is MCP? Model Context Protocol. The standard that lets AI agents connect to external data sources and tools. Like USB-C but for AI integrations. Atlan is an MCP server — Claude can query Atlan's metadata at runtime.

16. What is data governance? Rules about who can access data, who owns it, how it should be used. Critical for enterprises with compliance requirements like GDPR. Atlan enforces governance automatically.

17. What is data lineage? Tracking where data came from and where it goes. If a dashboard shows wrong numbers, lineage lets you trace it back to the source. Atlan captures this automatically.

Layer 4 — Builder Concepts (2 days)

For the technical round specifically.

18. What is an API? A way for two systems to talk to each other. You'll call the Anthropic or OpenAI API to use LLMs in code. REST APIs use HTTP requests — GET to fetch, POST to send.

19. What is a system prompt? Instructions you give an LLM before the conversation starts. Sets its behavior, persona, and constraints. Every AI product uses these heavily.

20. What is an evaluation (eval)? Testing whether an AI agent's output is correct or trustworthy. Atlan specifically mentioned "reliability & evaluation for AI agents" as a work area. Knowing this concept makes you stand out.

Resources for Each Layer

Layer 1 — LLMs & Agents

- "Intro to Large Language Models" by Andrej Karpathy on YouTube — best 1-hour explanation ever made
- "LLM Agents" by deeplearning.ai — free, takes 2 hours

Layer 2 — Data World

- sqlbolt.com — learn SQL interactively in one day
- "What is dbt?" on YouTube — any 10 min video works
- "Modern data stack explained" on YouTube

Layer 3 — AI meets Data

- Atlan's own blog covers all of this in readable form
- The reading list I gave you earlier covers this layer well

Layer 4 — Builder

- Anthropic docs at docs.anthropic.com — read the quickstart
- Build something tiny with the API — even just calling it from Python

LLM, context window, RAG, AI agent, function calling, hallucination, metadata, data lineage, MCP, and grounding.

The Scenario

Let's say a student uploads a complex engineering paper. Because of how the 500-word limit falls, a critical definition for a term like "Electromagnetic Field Theory" is split exactly in half.

- **Chunk A** contains the term.
- **Chunk B** contains the actual formula and explanation.

If the user asks a question about that formula, your vector search might only pull **Chunk B** because it has the keywords, but it might lack the "meaning" or "context" contained in **Chunk A**.

The Questions

1. **Technical Fix:** How would you modify your ingestion or retrieval logic to solve this "context fragmentation" problem?
2. **The Atlan Bridge:** Why is this problem of "fragmented context" such a big deal when Atlan is trying to help a company like **Mastercard** or **Nasdaq** close the "AI value chasm"?

How would you handle this "split context" issue to ensure the AI remains reliable, and how does that tie back to making sure a data team at a place like Nasdaq can trust their metadata?

The Technical Fix: "Sliding Window" Chunking

The solution is to use **Sliding Window Chunking with Overlap**.

- Instead of a hard cut every 500 words (e.g., 0–500, 501–1000), you create an **overlap**.
- For example, you might set a 100-word overlap. Chunk 1 would be words 0–500, and Chunk 2 would start at word 400.
- This ensures that the "context" at the boundaries of your notes is preserved in both chunks, so a definition is never lost

The Scenario

You are building an AI agent for a global enterprise like **Unilever**. They have thousands of documents indexed in a system similar to your **StudyMind** project. However, some of these documents are "Public," while others are "Restricted" (e.g., internal salary benchmarks or unreleased product designs).

The Questions

1. **Technical Implementation:** In your current StudyMind code (`ingest.py` or `agent.py`), how would you use **metadata** to ensure the AI agent only retrieves "Public" chunks for a general user, even if a "Restricted" chunk is semantically more similar to their question?
2. **Reliability & Evaluation:** Since Atlan emphasizes that "a wrong answer is worse than no answer," how would you use **AI-as-a-judge** to automatically test if your agent is staying within its "Public" boundaries?

Here is how you would handle that scenario using the logic from your preparation materials:

1. The Technical Fix: Metadata Filtering

In your project, metadata is "data about data," such as the filename or chunk number. To handle security policies, you would use **Metadata Filtering**:

- In `ingest.py`: When you call `collection.add()`, you would include a security label in the `metadatas` field for every chunk, such as `{"access_level": "public"}` or `{"access_level": "restricted"}`.
- In `agent.py`: When the user asks a question, you would modify the `collection.query()` call to include a "where" filter.

- **The Logic:** You would tell ChromaDB to only search for embeddings where the `access_level` is "public". This ensures that even if a "restricted" chunk is a perfect semantic match, the vector database physically blocks it from being retrieved.

2. The Reliability Fix: AI-as-a-Judge

Since Atlan believes "a wrong answer is worse than no answer," you need a way to prove your system is safe. You would implement an **Evaluation (Eval) System**:

- **The Judge LLM:** You would create a separate prompt for a "Judge" AI.
- **The Test:** You would give the Judge the user's question, the agent's final answer, and the metadata (citations) of the chunks used to generate that answer.
- **The Instruction:** You would ask the Judge: "Did the agent provide any information from a source labeled 'restricted' to this public user?".
- **The Goal:** This provides an automated "trust signal". If the Judge finds a violation, you flag it immediately.

3. The Atlan Bridge: Data Governance

This is the core of **Data Governance**—the rules about who can access what data and whether it is trustworthy.

- For an enterprise like **Unilever**, a "wrong" answer could mean leaking sensitive payroll data or trade secrets.
- Atlan acts as the "context and governance layer," ensuring that both humans and AI agents follow these security policies automatically.
- In your project, the source citation is like "lineage for knowledge"—it allows you to trace every answer back to its origin to verify it is authorized.

How to say it in the interview:

"I would use the `metadatas` field in `collection.add()` to tag chunks with access levels like 'public' or 'restricted'. Then, in my query logic, I'd apply a filter so the agent never even 'sees' restricted data during retrieval. To ensure this is reliable, I'd build an **Eval system** using an AI-as-a-judge to check that every answer only uses authorized citations, which maps directly to how Atlan manages data governance at scale."

The Questions

1. **Technical Implementation:** Why is your **RAG-based** architecture better for handling these frequent updates compared to a **fine-tuned** model? How would you modify your `ingest.py` or `agent.py` to ensure the database stays "fresh" and doesn't have stale or duplicate info?
2. **Observability & Trust:** Drawing from your experience building **health check and versioning endpoints** for the PSMRI project, how would you create a "Trust Signal" in your AI agent? How would the user know if the AI is reading the *latest* version of their data?

"I'd choose **RAG** over fine-tuning because it's the only way to handle frequently updating notes without the massive cost of retraining. To keep the data fresh, I'd use **metadata-based deletion** to clear old chunks and apply **conditional logic**—similar to what I used in my Traccar contribution—to only update if the file has actually changed. Finally, I'd build **observability** into the agent by including 'Last Updated' timestamps in the citations, which provides the 'Trust Signals' that enterprises like Nasdaq need to rely on AI-generated answers."

Bro this project is actually really impressive. It's more sophisticated than StudyMind in some ways — a live deployed API, Docker, Redis, state machine, and a genuinely smart architectural decision about AI reliability. Let me give you everything.

First — Understand Your Own Project Deeply

MerchantFlow is a **state-aware AI onboarding agent** for merchants. The core insight is brilliant and very Atlan-aligned:

"AI handles understanding. Backend handles execution."

Instead of letting the LLM decide what action to take — which causes hallucination — you use the LLM only to classify intent, then a strict state machine controls what actually happens. This is production-grade thinking.

ALL Questions and Answers

"Walk me through MerchantFlow."

"MerchantFlow is a merchant onboarding agent that guides merchants through three steps — GST verification, KYC verification, and bank verification — in strict order. The core architectural insight is that most AI agents fail in production because they let the LLM decide what to execute. That leads to hallucination and unpredictable behavior.

My approach is different. The LLM only classifies the user's intent — it reads the message and returns one of four labels: GST, KYC, BANK, or NEXT. That's it. A Spring Boot state machine then takes that label and decides what actually happens — enforcing the correct order, rejecting skipped steps with a 400 error, and tracking merchant progress in Redis. AI understands. Backend executes. No hallucination possible."

"What is a state machine and why did you use one?"

"A state machine is a system that can only be in one defined state at a time and moves between states through defined transitions. In MerchantFlow a merchant starts in STARTED state. After GST verification they move to GST_VERIFIED. Then KYC_DONE. Then BANK_VERIFIED. Then COMPLETED. You cannot jump from STARTED to KYC_DONE — the machine rejects invalid transitions.

I used it because onboarding has a strict business requirement — steps cannot be skipped. A state machine enforces this deterministically at the code level, not through AI judgment. That's far more reliable."

"Why did you use Redis for state storage?"

"Three reasons. Speed — Redis is an in-memory store, lookups are microseconds. A merchant sending 10 messages in a conversation shouldn't wait for database queries each time. Simplicity — storing and retrieving a merchant's current state is a simple key-value operation. Redis is perfect for that. Persistence across restarts — Redis persists state so if the server restarts, merchants don't lose their onboarding progress. The tradeoff is Redis stores data in memory so it's not ideal for complex relational queries — but for this use case it's exactly right."

"What is LangChain4j and why did you use it?"

"LangChain4j is the Java equivalent of LangChain — a framework that simplifies integrating LLMs into Java applications. It handles the API connection, prompt construction, and response parsing so I don't have to write that boilerplate manually. I used it because the project is Java-based and LangChain4j gave me a clean way to plug Gemini into a Spring Boot application without building the integration from scratch."

"Why did you use Spring Boot?"

"Spring Boot is the industry standard for building production Java APIs. It handles dependency injection, REST endpoint creation, error handling, and API documentation through SpringDoc automatically. For a merchant onboarding system that needs to be reliable and production-ready, Spring Boot was the right choice. It also gave me Swagger UI for free — which means anyone can test the API interactively without writing code."

"What is Docker and why did you containerize this?"

"Docker packages the application and all its dependencies into a container — an isolated, self-contained unit that runs identically anywhere. Without Docker, 'it works on my machine'

is a common problem. With Docker, the container runs exactly the same on my laptop, on a server, or on Render where I deployed it. Containerization also makes deployment simple — push the image, run the container, done. It's how most production backend systems are deployed today."

"How does the AI classification actually work?"

"The user sends a natural language message like 'verify my GST 27ABCDE1234F1Z4'. LangChain4j sends this to Gemini with a system prompt that says: classify this message into one of these four intents — GST, KYC, BANK, NEXT. Gemini returns a single label. The Spring Boot backend receives that label and looks up the merchant's current state in Redis. If the label is GST and the merchant is in STARTED state — proceed with GST verification. If the label is KYC but the merchant hasn't done GST yet — return a 400 error saying 'Please complete GST verification first'. The AI never knows about states or business logic. That knowledge lives in the backend."

"What happens if the AI misclassifies an intent?"

"That's one of the key strengths of this architecture. Even if Gemini misclassifies — say it labels a GST message as KYC — the state machine catches it. If the merchant is in STARTED state, a KYC intent will be rejected regardless of what the AI said. The state machine is the final authority, not the AI. This is why reliability improves dramatically when AI handles only understanding and backend handles execution."

"What is Swagger UI and why did you include it?"

"Swagger UI is an automatically generated interactive API documentation interface. It reads the API annotations in Spring Boot and creates a web page where anyone can see all endpoints, their parameters, and test them directly in the browser without writing any code. I included it because it makes the API immediately usable by anyone — no Postman setup, no reading documentation. Just open the URL and test it live."

"Why is this deployed on Render and not locally?"

"Deploying to production proves the project is real, not just a demo on my laptop. Render supports Docker deployments natively — I push the Docker image and it runs it on their servers with a public URL. It's free for small projects. The tradeoff is Render's free tier spins down inactive services, so the first request takes about 30 seconds to wake up. In production you'd use a paid tier or a different cloud provider to avoid that cold start."

"What is the difference between this project and a normal chatbot?"

"A normal chatbot lets the LLM decide everything — what action to take, what order to do things in, what response to give. That's fine for casual conversation but dangerous for business workflows where order and correctness matter. MerchantFlow is not a chatbot — it's a workflow engine with a conversational interface. The LLM is just a translator between human language and machine labels. The workflow engine is in complete control. This distinction is exactly what separates AI toys from production AI systems."

"How does this relate to what Atlan builds?"

"The parallel is direct. Atlan's AI agents operate on enterprise data — and a wrong action on enterprise data is catastrophic. Atlan solves this by grounding AI agents in a context layer so they reason on trusted, governed data rather than making things up. MerchantFlow solves the same reliability problem differently — by restricting AI to classification only and letting deterministic backend logic handle execution. Both approaches share the same philosophy: AI is powerful but needs guardrails to be production-safe. The guardrail in Atlan is the context layer. The guardrail in MerchantFlow is the state machine."

"What would you improve if you had more time?"

"Four things from my own roadmap. First — JWT authentication so only authorized merchants can access their own onboarding data. Right now any GST number can be queried. Second — PostgreSQL for persistent merchant data instead of just Redis, because Redis alone isn't suitable for long-term storage. Third — async processing for verification steps, because in a real system GST and KYC verification would involve external API calls that take time. Fourth — multi-turn conversation memory so the agent remembers context across a full conversation, not just a single message."

"What is Maven and what does pom.xml do?"

"Maven is a build tool for Java projects. It manages dependencies — like LangChain4j, Spring Boot, Redis client — by listing them in pom.xml. When you run `mvn clean package`, Maven downloads all dependencies, compiles the code, runs tests, and packages everything into a runnable JAR file. The pom.xml is essentially the recipe for building the project — which libraries to include, which Java version to use, and how to package the output."

"How did you test this project?"

"Two ways. First — Swagger UI. I used the live interactive interface to manually test every endpoint and every state transition. I verified that skipping steps correctly returns 400 errors and that valid sequences complete successfully. Second — live demo deployment on

Render. Having a real deployed URL meant I could test from any device and share it with others. What I'd add next is automated integration tests using Spring Boot Test that verify every state machine transition automatically on every code change."

System Design Questions

"How would you scale MerchantFlow to handle 1 million merchants?"

"Several changes. First — Redis would need to be clustered, not a single instance. Redis Cluster distributes data across multiple nodes so one node failure doesn't lose all state. Second — the Spring Boot backend would need to be horizontally scaled — multiple instances behind a load balancer. Since state is in Redis and not in application memory, any instance can serve any merchant. Third — the Gemini API calls would need rate limiting and queuing — at high volume you'd hit API limits fast. A message queue like Kafka would buffer requests. Fourth — database — I'd add PostgreSQL for permanent merchant records. Redis holds current state. PostgreSQL holds history and audit trail."

"How would you add real GST and KYC verification instead of mock verification?"

"Replace the mock verification logic with real external API calls. For GST — India's GST API is publicly available and lets you validate GST numbers. For KYC — third-party providers like DigiLocker or Aadhaar APIs handle document verification. The state machine wouldn't change at all — it just waits for a success or failure response from the external API before transitioning state. I'd add retry logic for failed API calls and timeout handling so a slow external API doesn't hang the merchant's session."

"How would you add conversation memory so the agent remembers context?"

"Two approaches. Simple — store the last N messages in Redis alongside the merchant state, and include them in every Gemini prompt as conversation history. This gives the AI context about previous messages. Advanced — use a vector database to store conversation history as embeddings, then retrieve relevant past messages based on semantic similarity to the current message. The simple approach works for short conversations. The vector approach works for long conversations where you can't fit everything in the context window."

The Story to Tell

This project has a better story than most intern projects because it's **actually deployed and live**. Use that.

"MerchantFlow is live at a public URL right now. You can hit the Swagger UI and test it in real time. It handles real HTTP requests, stores state in Redis, classifies intent with Gemini, and enforces workflow with a state machine. The core insight I wanted to prove was that the most reliable AI systems aren't the ones with the smartest AI — they're the ones where AI does only what AI is good at. Classification. Understanding. Not execution. Not decision-making. That boundary is what makes it production-safe. And that's exactly the philosophy Atlan applies — AI agents need governed, grounded context before they act. My state machine is my version of that governance layer."

That story is genuinely strong. 🚀

1. "Tell me about yourself and why you applied."

"I'm Vaishnav, a B.Tech student graduating in 2027. I'm someone who learns by building — not by reading or watching tutorials. When I got this opportunity I had zero knowledge of Atlan's domain. Instead of just reading about it, I spent the last few weeks actually building two projects that demonstrate the core problem Atlan solves — reliable AI on trusted context.

The reason I applied specifically to Atlan is this one line from your website: 95% of AI pilots fail not because the models are bad — but because AI doesn't understand the context behind data. That sentence reframed everything for me. Everyone is racing to build better models. Nobody is building the infrastructure that makes those models actually trustworthy inside enterprises. Atlan is. That's the problem I want to work on."

2. "Walk me through StudyMind."

"StudyMind came from a real problem I faced while preparing for this internship. I had lecture notes, PDFs, and research material scattered everywhere — but no way to query across them. When I asked ChatGPT it would give me confident answers that weren't actually in my material. That's hallucination — and it's dangerous.

Then I realized — this is exactly what Atlan solves for enterprises. Data teams have the same problem. Data scattered across warehouses and dashboards, AI agents reasoning on missing or unverified context, no single trusted source of truth.

So I built StudyMind — a personal context layer for students. You drop your PDFs into a folder. The system extracts text, splits it into 500-word chunks, converts each chunk into an embedding using Gemini's embedding model, and stores everything in ChromaDB — a local vector database. When you ask a question, it converts your question into an embedding too, finds the most similar chunks using vector similarity search, and passes only those chunks to Gemini as context. Gemini answers only from that context — with source citations showing exactly which PDF the answer came from.

The result is zero hallucination — architecturally. Not because I told the AI to be careful. Because it literally cannot see anything except your actual notes.

The hardest part wasn't the AI. It was the data ingestion layer — PDF extraction is messy, chunking affects answer quality, and a library I was using got deprecated mid-build. That taught me the real lesson — the integration layer is always harder than it looks. Which is probably true at Atlan's scale too."

3. "Walk me through MerchantFlow."

"MerchantFlow is a merchant onboarding agent built in Java with Spring Boot. A merchant gets onboarded through three steps — GST verification, KYC verification, and bank verification — in that strict order.

The core insight behind the project is this: most AI agents fail in production because they let the LLM decide what action to execute. That leads to hallucination, skipped steps, and unpredictable behavior. My approach is different.

AI handles understanding. Backend handles execution.

The LLM — connected through LangChain4j and Gemini — reads the merchant's message and classifies intent into one of four labels: GST, KYC, BANK, or NEXT. That's all it does. A Spring Boot state machine then takes that label and decides what actually happens — enforcing the correct order, rejecting skipped steps with a 400 error, tracking each merchant's progress in memory.

So even if the AI misclassifies — say it reads a GST message as KYC — the state machine catches it. If the merchant hasn't completed GST yet, the KYC intent is rejected regardless of what the AI said. The state machine is the final authority, not the LLM.

It's deployed live on Render with Docker. Anyone can test it right now through Swagger UI — create a merchant, try to skip a step, see the 400 error, complete the steps in order.

The governance layer in MerchantFlow is the state machine. The governance layer in Atlan is the context layer. Different implementation, same philosophy — AI needs deterministic guardrails to be production-safe."

4. "How do these projects relate to what Atlan does?"

"Both projects are small-scale versions of the exact problem Atlan solves at enterprise scale.

Atlan's problem: enterprises have data scattered across Snowflake, dbt, Tableau, Airflow — with no unified layer that tells AI agents what that data means, who owns it, whether to trust it, and how to use it. Without that context layer, AI agents hallucinate, make wrong decisions, and can't be trusted in production.

StudyMind's version of the same problem: a student has notes scattered across PDFs with no unified layer that lets AI query across them reliably. Without grounding, the AI hallucates. With a context layer — ChromaDB plus source citations — every answer is trusted and traceable. That's lineage for student knowledge.

MerchantFlow's version: an onboarding workflow where AI needs to be reliable and deterministic. Without guardrails, AI agents skip steps and cause errors. With a state machine as the governance layer, wrong actions are architecturally impossible.

The pattern across both projects is identical to Atlan's pattern — trusted context plus deterministic governance equals reliable AI. Atlan does this for Mastercard's 18 million data assets. I did it for a student's 10 PDFs and a merchant's three onboarding steps. Same architecture, different scale."

5. "What do you not know yet that matters for this role?"

"Honestly — quite a bit, and I'd rather be direct about it than pretend otherwise.

I haven't worked with dbt, Snowflake, or Databricks at all. These are core to the data stack Atlan integrates with, and I'd need to get up to speed on them quickly after joining. My SQL knowledge is basic — I understand queries conceptually but haven't used it extensively in real projects. I've never shipped code in a real engineering team with code reviews, PRs, and production deployments at scale. And my Python experience came largely from building StudyMind — it's still early.

What I do have is the ability to learn fast by building. Everything I know about RAG, vector databases, state machines, and AI reliability — I learned by building two working projects in a few weeks with no prior knowledge of the domain. I didn't wait to understand everything before starting. I started, broke things, fixed them, and understood by doing.

I think that learning pattern matters more at this stage than any specific checklist of technologies. And I'm genuinely excited about the gaps — because closing them at Atlan means learning in the best possible environment."

The Golden Rule

These answers are longer written down than they need to be spoken. When you say them out loud they'll naturally be tighter and more conversational. Don't try to fit every word in — hit the key ideas and let the conversation flow.

The structure that matters in each answer:

Tell me about yourself — who I am → how I learn → why this problem specifically

StudyMind — real problem → realization → how it works → hardest part → lesson learned

MerchantFlow — core insight → how it works → why the architecture → live demo → connection to Atlan

How they relate — Atlan's problem → StudyMind's version → MerchantFlow's version → the pattern

What you don't know — specific honest gaps → what you do have → learning pattern → excitement not fear

Say these out loud three times each before the interview. Not to memorize — to make them feel natural. 🚀